

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive")

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
```

```
file = '/content/drive/MyDrive/B.TECH/5th Sem/MDM Dataset/wireless_signal_quality_dataset.xlsx - Sheet1.csv'
```

```
file = '/content/wireless_signal_quality_dataset.xlsx - Sheet1.csv'
data = pd.read_csv(file)
data.info(), data.head(), data.tail()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 100 entries, 0 to 99
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	Signal_Strength_dBm	100 non-null	int64
1	SNR_dB	100 non-null	int64
2	Interference_dB	100 non-null	int64
3	Distance_m	100 non-null	int64
4	Frequency_GHz	100 non-null	float64
5	Channel_Bandwidth_MHz	100 non-null	int64
6	Noise_Floor_dBm	100 non-null	int64
7	Connected_Devices	100 non-null	int64
8	Estimated_Throughput_Mbps	100 non-null	int64
9	Quality	100 non-null	object

```
dtypes: float64(1), int64(8), object(1)
```

```
memory usage: 7.9+ KB
```

```
(None,
```

	Signal_Strength_dBm	SNR_dB	Interference_dB	Distance_m	Frequency_GHz	\
0	-41	6	0	257	5.0	
1	-86	25	20	134	2.4	
2	-73	27	0	385	2.4	
3	-57	5	0	255	5.0	
4	-87	15	24	61	5.0	

	Channel_Bandwidth_MHz	Noise_Floor_dBm	Connected_Devices	\
0	160	-100	24	
1	160	-96	41	
2	20	-72	13	
3	160	-86	9	
4	20	-79	32	

	Estimated_Throughput_Mbps	Quality
0	18	Poor
1	55	Poor
2	81	Fair
3	15	Poor
4	21	Poor

	Signal_Strength_dBm	SNR_dB	Interference_dB	Distance_m	Frequency_GHz	\
95	-75	37	11	17	2.4	
96	-88	11	18	231	5.0	
97	-57	6	27	460	5.0	
98	-49	25	24	298	2.4	
99	-46	8	29	423	5.0	

	Channel_Bandwidth_MHz	Noise_Floor_dBm	Connected_Devices	\
95	160	-80	34	
96	20	-90	25	
97	160	-76	24	
98	20	-78	39	

```

99          40          -71          16

Estimated_Throughput_Mbps Quality
95          100    Fair
96          15    Poor
97           1    Poor
98          51    Fair
99          1    Poor

```

```
data.isnull().sum()
```

```

Signal_Strength_dBm    0
SNR_dB                 0
Interference_dB        0
Distance_m             0
Frequency_GHz          0
Channel_Bandwidth_MHz  0
Noise_Floor_dBm        0
Connected_Devices      0
Estimated_Throughput_Mbps 0
Quality                0

```

```
dtype: int64
```

```
print(data.isnull().sum())
```

```

Signal_Strength_dBm    0
SNR_dB                 0
Interference_dB        0
Distance_m             0
Frequency_GHz          0
Channel_Bandwidth_MHz  0
Noise_Floor_dBm        0
Connected_Devices      0
Estimated_Throughput_Mbps 0
Quality                0
dtype: int64

```

```

label_encoder = LabelEncoder()
data['Quality'] = label_encoder.fit_transform(data['Quality'])

```

```

X = data.drop('Quality', axis=1)
y = data['Quality']

```

```
X = pd.get_dummies(X, drop_first=True)
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

```

```

#Train split
X_train,X_test,Y_train,Y_test=train_test_split(
    X,y,
    test_size=0.2,

```

```
random_state=42
)
```

```
#Feature scaling
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
```

```
#Logistic Regression
model=LogisticRegression(max_iter=1800)
model.fit(X_train,Y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1800)
```

```
# Prediction
Y_pred=model.predict(X_test)
```

```
#Evaluation
print("\nAccuracy:")
print(accuracy_score(Y_test,Y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(Y_test,Y_pred))
print("\nClassification Report:")
print(classification_report(Y_test,Y_pred))
```

Accuracy:
0.65

Confusion Matrix:
[[0 0 1 0]
[0 4 0 2]
[0 2 1 0]
[0 2 0 8]]

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1
1	0.50	0.67	0.57	6
2	0.50	0.33	0.40	3
3	0.80	0.80	0.80	10
accuracy			0.65	20
macro avg	0.45	0.45	0.44	20
weighted avg	0.62	0.65	0.63	20

```
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarnin
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarnin
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarnin
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

